

**EXP 4: IMPLEMENT BFT AND DFT FOR GIVEN GRAPH, WHEN GRAPH IS
REPRESENTED BY
ADJACENCY MATRIX AND ADJACENCY LISTS**

4(i): Adjacency Matrix

AIM:

To implement Breadth First Traversal (BFT) and Depth First Traversal (DFT) for a graph represented using an Adjacency Matrix.

PROCEDURE:

1. Start the program.
2. Read the number of vertices.
3. Read the adjacency matrix of the graph.
4. Take 'A' as the starting vertex.
5. Convert the starting vertex A into index 0.
6. For BFT:
 - o Initialize a queue.
 - o Insert vertex A into the queue and mark it visited.
 - o Delete a vertex from the queue and visit it.
 - o Insert all its unvisited adjacent vertices into the queue.
 - o Repeat until the queue is empty.
7. For DFT:
 - o Initialize a stack.
 - o Push vertex A onto the stack.
 - o Pop a vertex and visit it if it is unvisited.
 - o Push all its unvisited adjacent vertices onto the stack.
 - o Repeat until the stack is empty.
8. Display the BFT and DFT traversal order.
9. Stop the program.

PROGRAM:

```
#include <stdio.h>
```

```
void bft(int graph[10][10], int n, int start)
{
    int queue[10], front = 0, rear = 0;
    int visited[10] = {0};
    int i, v;

    queue[rear++] = start;
    visited[start] = 1;

    printf("\n BFT TRAVERSAL : ");
```

```

while (front < rear)
{
    v = queue[front++];
    printf("%c ", v + 'A');

    for (i = 0; i < n; i++)
    {
        if (graph[v][i] == 1 && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
}

}

void dft(int graph[10][10], int n, int start)
{
    int stack[10], top = -1;
    int visited[10] = {0};
    int i, v;

    stack[++top] = start;

    printf("\nDFT TRAVERSAL: ");

    while (top != -1)
    {
        v = stack[top--];

        if (visited[v] == 0)
        {
            printf("%c ", v + 'A');
            visited[v] = 1;

            for (i = n - 1; i >= 0; i--)
            {
                if (graph[v][i] == 1 && visited[i] == 0)
                    stack[++top] = i;
            }
        }
    }
}

int main()
{

```

```

int graph[10][10];
int n, i, j, start;
char ch;

printf("Enter number of vertices: ");
scanf("%d", &n);

printf("Enter adjacency matrix:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &graph[i][j]);
    }
}
printf("Enter starting vertex (A-%c): ", 'A' + n - 1);
scanf(" %c", &ch);
start = ch - 'A';

bfs(graph, n, start);
dfs(graph, n, start);

return 0;
}

```

OUTPUT:

D:\>gcc AdjMatrix.c -o AdjMatrix.exe

D:\>AdjMatrix.exe

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

Enter starting vertex (A-D): A

BFT TRAVERSAL : A B C D

DFT TRAVERSAL: A B C D

RESULT:

Thus BFT and DFT of the given graph using an Adjacency Matrix were successfully implemented.

4(ii): Adjacency Lists

AIM:

To implement Breadth First Traversal (BFT) and Depth First Traversal (DFT) for a graph represented using an Adjacency Lists.

PROCEDURE:

1. Start the program.
2. Read the number of vertices and edges.
3. Enter the edges of the graph using vertex names such as A, B, C, D.
4. Create the Adjacency List for the graph.
5. For BFT
 - Start BFT from vertex 'A'.
 - Insert 'A' into the Queue and mark it as visited.
 - Remove a vertex from the queue and visit its unvisited adjacent vertices.
 - Repeat until the queue becomes empty.
6. For DFT
 - Start DFT from vertex 'A'.
 - Insert 'A' into the Stack and mark vertices as visited.
 - Pop a vertex from the stack and visit its unvisited adjacent vertices.
 - Repeat until the stack becomes empty.
7. Display the BFT and DFT traversal orders.
8. Stop the program

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
{
    int vertex;
    struct node *next;
};
```

```
struct node *adj[10];
```

```
struct node * createNode(int v)
{
```

```

    struct node *p = (struct node *)malloc(sizeof(struct node));
    p->vertex = v;
    p->next = NULL;
    return p;
}

```

```

/* BFT using Queue */
void bft(int n)
{
    int queue[10], front = 0, rear = 0;
    int visited[10] = {0};
    int v, i;
    struct node *p;

    queue[rear++] = 0;    // Start from A
    visited[0] = 1;

    printf("\nBFT: ");

    while(front < rear)
    {
        v = queue[front++];
        printf("%c ", v + 'A');

        p = adj[v];

        while(p != NULL)
        {
            i = p->vertex;

            if(visited[i] == 0)
            {
                queue[rear++] = i;
                visited[i] = 1;
            }

            p = p->next;
        }
    }
}

```

```

/* DFT using Stack */
void dft(int n)
{
    int stack[10], top = -1;
    int visited[10] = {0};

```

```

int v, i;
struct node *p;

stack[++top] = 0;    // Start from A

printf("\nDFT: ");

while(top != -1)
{
    v = stack[top--];

    if(visited[v] == 0)
    {
        printf("%c ", v + 'A');
        visited[v] = 1;

        p = adj[v];

        while(p != NULL)
        {
            i = p->vertex;

            if(visited[i] == 0)
                stack[++top] = i;

            p = p->next;
        }
    }
}

int main()
{
    int n, e, i;
    char s, d;
    int src, dest;
    struct node *p;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        adj[i] = NULL;

    printf("Enter number of edges: ");
    scanf("%d", &e);

```

```

printf("Enter edges:\n");

for(i = 0; i < e; i++)
{
    scanf(" %c %c", &s, &d);

    src = s - 'A';
    dest = d - 'A';

    /* src -> dest */
    p = createNode(dest);
    p->next = adj[src];
    adj[src] = p;

    /* dest -> src */
    p = createNode(src);
    p->next = adj[dest];
    adj[dest] = p;
}

printf("\nAdjacency List:\n");

for(i = 0; i < n; i++)
{
    printf("%c -> ", i + 'A');

    p = adj[i];

    while(p != NULL)
    {
        printf("%c ", p->vertex + 'A');
        p = p->next;
    }

    printf("\n");
}

bft(n);
dft(n);

return 0;
}

```

OUTPUT:

D:\>gcc AdjList.c -o AdjList.exe

D:\>AdjList.exe

Enter number of vertices: 4

Enter number of edges: 5

Enter edges:

A B

A C

A D

B C

C D

Adjacency List:

A -> D C B

B -> C A

C -> D B A

D -> C A

BFT: A D C B

DFT: A B C D

RESULT:

Thus BFT and DFT of the given graph using an Adjacency Lists were successfully implemented.